Protocols, cryptanalysis and mathematical cryptology (INFO-F514)

Homomorphic Encryption

Christophe Petit

Université libre de Bruxelles

Sensitization campaign

Together against sexual harassment and violences
https://www.ulb.be/fr/sante-et-bien-etre/si-cest-pas-oui-cest-non

Protocols, cryptanalysis and mathematical cryptology (INFO-F514)

Homomorphic Encryption

Christophe Petit

Université libre de Bruxelles

Computing on encrypted data

- Motivation: in e-voting and other applications, apparent contradiction between
  - Data should be encrypted due to privacy concerns
  - Data needs to be manipulated to be useful
- Homomorphic encryption: given $Enc(m_1)$ and $Enc(m_2)$, can compute $Enc(m_1 \oplus m_2)$ for some operation $\oplus$
- Fully homomorphic encryption: encryption scheme homomorphic with respect to a *complete* set of operations (for example, addition and multiplication)

Computing on encrypted data

- How could this be useful for e-voting applications?

Outline

Definition and properties

Multiplicatively homomorphic cryptosystems

Additively homomorphic cryptosystems

Application to e-voting

Somewhat fully homomorphic encryption

Fully homomorphic encryption

Outline

Definition and properties

Multiplicatively homomorphic cryptosystems

Additively homomorphic cryptosystems

Application to e-voting

Somewhat fully homomorphic encryption

Fully homomorphic encryption

Reminder: Public Key Encryption Scheme

- Key Generation algorithm **KeyGen**
  - Given security parameter λ
  - Return pair of private, public keys (SK, PK)
- Encryption algorithm **Enc**
  - Given public key PK and message m
  - Return ciphertext c
- Decryption algorithm **Dec**
  - Given private key SK and ciphertext c
  - Return message m

Homomorphic encryption

- Let $\oplus$ be an operation on plaintexts, and $\otimes$ be an operation on ciphertexts
- We say the encryption scheme is homomorphic with respect to those operations if

$$\mathbf{Enc}(m_1 \oplus m_2) = \mathbf{Enc}(m_1) \otimes \mathbf{Enc}(m_2)$$

- We say the encryption scheme is additively (resp. multiplicatively) homomorphic if $\oplus$ is addition (resp. multiplication) (both typically over finite structure)

Reminder: IND-CPA security

- Indistinguishability against chosen plaintext attacks
- Security game
  - Challenger runs $(PK, SK) \leftarrow \mathbf{KeyGen}(\lambda)$
  - Challenger sends PK to adversary
  - Adversary chooses two messages m_0, m_1
  - Adversary sends m_0, m_1 to challenger
  - Challenger picks a random bit $b \leftarrow \{0, 1\}$
  - Challenger sends $c = \mathbf{Enc}(PK, m_b)$ to adversary
  - Adversary sends a bit b' to challenger
  - Adversary wins if $b' = b$
- An encryption scheme is IND-CPA secure if

$$|\Pr[\text{Adversary wins}] - 1/2| \approx 0$$

Reminder: IND-CCA(2) security

- IND-CCA(2) = indistinguishability against (adaptively) chosen ciphertext attacks
- Adversary is given an additional power: decryption oracle
  - Before sending m_0, m_1, adversary can call a decryption oracle on ciphertexts of its choice
  - In adaptive version, can also call the decryption oracle after receiving c and before sending b' (adversary not allowed to call oracle on c)

Homomorphic $\Rightarrow$ not IND-CCA2 secure

- In additive case (general case similar)
  - Adversary sends arbitrary messages m_0, m_1
  - Adversary receives $c_b = \mathbf{Enc}(m_b)$ for randomly chosen b
  - Adversary computes $c'_b = c_b \otimes \mathbf{Enc}(0)$
  - Adversary asks oracle to decrypt c'_b and receives m_b: indeed
    $c'_b = \mathbf{Enc}(m_b) \otimes \mathbf{Enc}(0) = \mathbf{Enc}(m_b + 0) = \mathbf{Enc}(m_b)$

Outline

Definition and properties

Multiplicatively homomorphic cryptosystems

Additively homomorphic cryptosystems

Application to e-voting

Somewhat fully homomorphic encryption

Fully homomorphic encryption

The RSA ring

- Let p, q be two primes and let $n = pq$
- Let $\mathbb{Z}_n := \mathbb{Z}/n\mathbb{Z}$ be the ring of integers modulo n
- Let $\varphi(n) := (p-1)(q-1) = |\mathbb{Z}_n^*|$
- Euler's theorem: for all $x \in \mathbb{Z}_n^*$, we have

$$x^{\varphi(n)} = 1 \bmod n$$

RSA encryption

- Key generation:
  - Generate two *strong* primes p and q of same size, size given by the security parameter
  - Compute $n = pq$
  - Generate random $e \in [1, \varphi(n)]$ with $\gcd(e, \varphi(n)) = 1$
  - Compute d such that $ed = 1 \bmod \varphi(n)$
  - Return private key (p, q, d) and public key (n, e)
- Encryption: given a message m and public key (n, e), return $c = m^e \bmod n$
- Decryption: given ciphertext c and private key (p, q, d), return $m' = c^d \bmod n$

RSA security

- Computing private key from public key $\Leftrightarrow$ factoring n
- Decrypting a ciphertext without the private key requires computing e-th roots modulo n, a problem often called the RSA problem
- Textbook RSA is deterministic, hence not IND-CPA, hence not IND-CCA, hence not IND-CCA2
- Weaknesses overcome in variants including RSA-OAEP

RSA is multiplicatively homomorphic

- Let $c_1 = m_1^e \bmod n$ encryption of m_1
- Let $c_2 = m_2^e \bmod n$ encryption of m_2
- Then $c_1 c_2 \bmod n$ is an encryption of $m_1 m_2 \bmod n$

- Typically another reason not to use Textbook RSA and use RSA-OAEP instead
- Unless you want homomorphic property !

ElGamal encryption

- Public key is cyclic group G, generator g and $h = g^s$
- Private key is s
- Encryption of $m \in G$ is $(c_1, c_2) = (g^r, mh^r)$ for some random r
- To decrypt (c_1, c_2) compute $c_2/c_1^s = m$

Homomorphic properties of ElGamal

- Let $(c_1, c_2) = (g^r, mh^r)$ encryption of m
- Let $(c_1', c_2') = (g^{r'}, m'h^{r'})$ encryption of m'
- Then

$$(C_1, C_2) = (c_1 c_1', c_2 c_2') = (g^{r+r'}, mm'h^{r+r'})$$

  is an encryption of mm'

- If $m' = 1$ then (C_1, C_2) is a new encryption of m
- Use Cramer-Shoup variant for IND-CCA security

Outline

Definition and properties

Multiplicatively homomorphic cryptosystems

Additively homomorphic cryptosystems

Application to e-voting

Somewhat fully homomorphic encryption

Fully homomorphic encryption

Exponential ElGamal

- Public key is cyclic group G, generator g and $h = g^s$, bound B
- Private key is s
- Encryption of $0 \leq m < B$ is $(c_1, c_2) = (g^r, g^m h^r)$ for some random r
- To decrypt (c_1, c_2) first compute $c_2/c_1^s = g^m$, then solve DLP over a small interval to get m
- DLP feasible over small intervals (exhaustive search, baby-step-giant-step methods)
- Same security as ElGamal (prove it!)

Homomorphic properties of Exponential ElGamal

- Let $(c_1, c_2) = (g^r, g^m h^r)$ encryption of m
- Let $(c_1', c_2') = (g^{r'}, g^{m'} h^{r'})$ encryption of m'
- Then

$$(C_1, C_2) = (c_1 c_1', c_2 c_2') = (g^{r+r'}, g^{m+m'} h^{r+r'})$$

  is an encryption of $m + m'$ (modulo group order)

- If $m' = 0$ then (C_1, C_2) is a new encryption of m

Paillier cryptosystem: key generation

- Let p, q be large primes and $n = pq$ as in RSA
- Consider group $\mathbb{Z}^*_{n^2}$ and its nth roots of unity
  - Given by $x_k := (1+n)^k = 1 + kn \bmod n^2$ for all k
  - Let $L : x_k \to k = \frac{x_k - 1}{n}$
  - DLP is easy in subgroup:
    $L(x_k^m) = L((1+n)^{km}) = km = mL(x_k)$
- Let $\lambda = \lambda(n) = \mathrm{lcm}(p-1, q-1)$
  - For any $g \in \mathbb{Z}^*_{n^2}$, element g^λ is a nth root of unity
  - Let $g \in \mathbb{Z}^*_{n^2}$ random with order divisible by n
  - Let $\mu = (L(g^\lambda \bmod n^2))^{-1} \bmod n$
- Public key is (n, g) and secret key is (λ, μ)

Paillier cryptosystem: encryption and decryption

- Encryption
  - Let $0 \leq m < n$ be a message to be encrypted
  - Let $r \in \mathbb{Z}_n^*$
  - Let $c = g^m r^n \bmod n^2$
- Decryption: let $m = \mu \cdot L(c^\lambda \bmod n^2) \bmod n$
- Intuitions:
  - DLP easy for nth roots of unity (with function L)
  - Powering g^m by λ gives you nth root of unity
  - Multiplication by r^n brings in some randomness, disappears when powering by λ

Paillier cryptosystem: properties

- Correctness
  - There exists k such that $g^\lambda = 1 + kn \bmod n^2$
  - $c^\lambda = g^{m\lambda} r^{\lambda n} = g^{m\lambda} = (1+kn)^m = 1 + kmn \bmod n^2$
  - $\mu L(c^\lambda) = \frac{L(c^\lambda)}{L(g^\lambda)} = \frac{km}{k} = m$
- Paillier cryptosystem IND-CPA secure if composite residuosity assumption holds: given $z \in \mathbb{Z}^*_{n^2}$, hard to decide whether there is $y \in \mathbb{Z}^*_{n^2}$ such that $z = y^n \bmod n^2$ (i.e. whether z in the subgroup of order λ)
- Homomorphic properties: $\mathbf{Enc}(m_1) \cdot \mathbf{Enc}(m_2) = g^{m_1} r_1^n \cdot g^{m_2} r_2^n = g^{m_1+m_2}(r_1 r_2)^n = \mathbf{Enc}(m_1 + m_2)$

Outline

Definition and properties

Multiplicatively homomorphic cryptosystems

Additively homomorphic cryptosystems

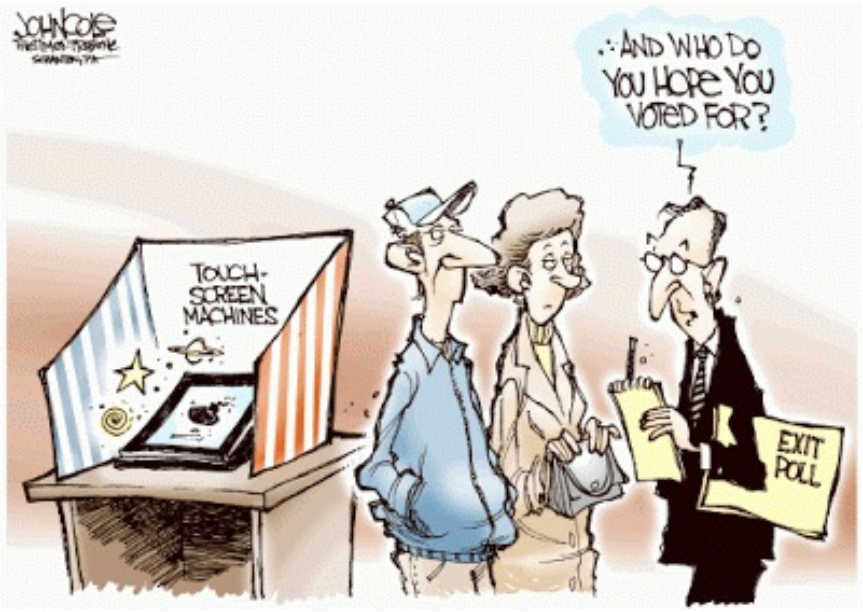
Application to e-voting

Somewhat fully homomorphic encryption

Fully homomorphic encryption

Electronic elections

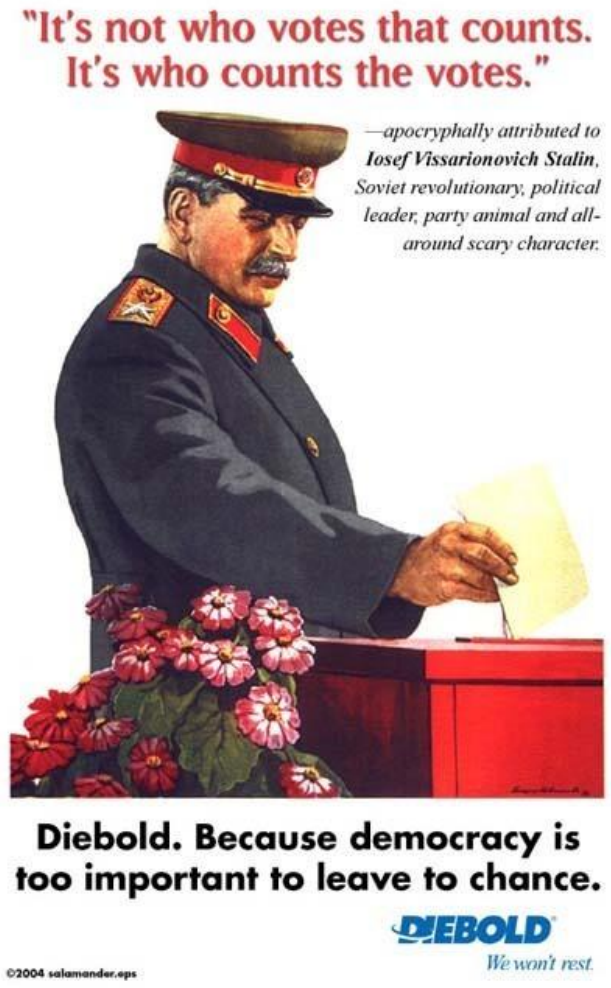
In the News:
Michigan Voters
with Blindness
Prefer ES&S



Kids at hacking conference show how easily US elections could be sabotaged

Headlines from: Election Systems & Software, retrieved 15/11/2018; The Guardian, 22/08/2018.
Cartoon by John Cole published on `e-lected.blogspot.com`. Picture of the Diebold AccuVote-TS.

DRE questionned



https://columbusfreepress.com/article/diebold-indicted-its-spectre-still-haunts-ohio-elections

Cryptography to the rescue

- Cryptographers designed **end-to-end verifiable** voting systems, where voters can verify that their votes are
  - Cast as intended
  - Recorded as cast
  - Counted as recorded
- Now used for general elections in Estonia and more widely for elections with low coercion threat level

e-voting attempt

- Election officials generate public key encryption key
- Each voter encrypts their vote with the public key; vote is sent to election officials
- Each vote is decrypted with secret key and tallied
- Issues?
  - Election officials can know all the votes
  - No guarantee they do not add, remove or change votes
  - Users can prove how they voted

Improvement: publishing encrypted votes

- Encrypted votes published on a bulletin board
- Voters can verify their vote is on the bulletin board
- Tallying process can be witnessed to ensure no vote is added/changed compared to bulletin board
- Each vote can also be signed by voter; in which case everybody can check their integrity on the bulletin board (but not possible deletions)
- Remaining issues?
  - Election officials and witness can know all the votes
  - Users can prove how they voted

Two approaches to e-voting

- Homomorphic encryption: use encryption scheme such that
$$Enc(v_1 + v_2) = Enc(v_1) \otimes Enc(v_2)$$
for some operation $\otimes$,
  $\rightarrow$ Only end result needs to be decrypted

- Mixnets: shuffle the votes before decryption
  $\rightarrow$ Votes can no longer be linked to voters
  $\rightarrow$ Votes can be decrypted individually

e-voting attempt: homomorphic encryption

- For each candidate, election officials generate keys for Exponential ElGamal encryption
- Voters encrypt 1 if they want to vote for a candidate; they encrypt 0 otherwise
- Votes are published
- Encryption of result can be computed by anyone
- Election officials decrypt result

- Issues?
  - No guarantee that votes are well-formed (e.g. encryption of 1000 or -1)
  - Election officials can also decrypt individual votes
  - Users can prove how they voted

e-voting attempt: homomorphic encryption

- For each candidate, election officials generate keys for **treshold** exponential ElGamal encryption
- Voters encrypt 1 if they want to vote for a candidate; they encrypt 0 otherwise; **they append a proof of this**
- Votes are published
- Correctness of votes can be checked by anyone
- Encryption of result can be computed by anyone
- **Election officials must cooperate to decrypt result; we assume they will not cooperate to decrypt individual votes**
- Elections officials publish **proof** that decryption correct
- Users can still prove how they voted

Another e-voting attempt

- Election official generates encryption keys
- Voters send their encrypted votes to trustee
- Trustee takes all votes, re-randomizes and shuffles them
- Election official takes shuffled votes, decrypts, computes result, proves result

- Randomization and shuffling prevents election official from linking individual votes to voter
- Randomization must be done on encrypted data

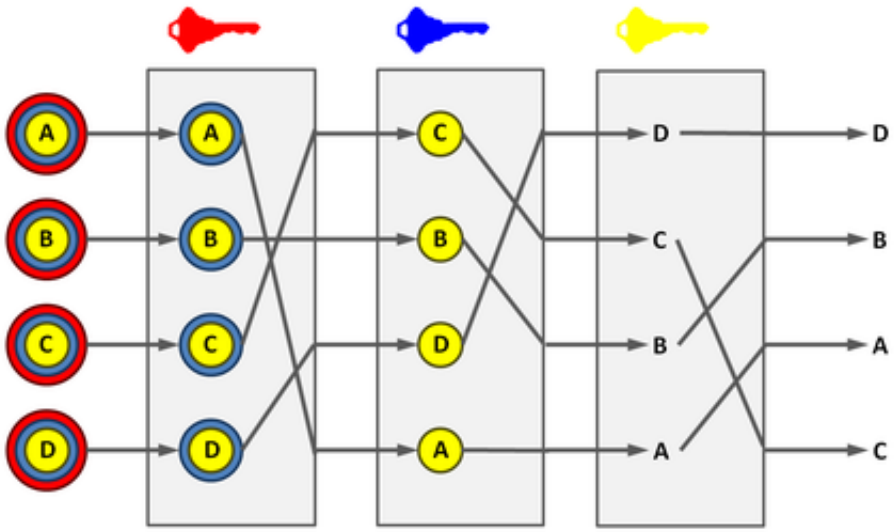
Homomorphic properties of Exponential ElGamal

- Let $(c_1, c_2) = (g^r, g^m h^r)$ encryption of m
- Let $(c_1', c_2') = (g^{r'}, g^{m'} h^{r'})$ encryption of m'
- Then

$$(C_1, C_2) = (c_1 c_1', c_2 c_2') = (g^{r+r'}, g^{m+m'} h^{r+r'})$$

  is an encryption of $m + m'$ (modulo group order)
- If $m' = 0$ then (C_1, C_2) is a new encryption of m
  $\rightarrow$ gives a re-randomization procedure!

Mixnets



- Each trustee generates their keys
- Each vote is encrypted several times, one per trustee
- Each trustee removes one encryption layer and shuffles all the votes; they also publish a **proof** that they did their computation properly

Picture credit: Wikipedia

Outline

Definition and properties

Multiplicatively homomorphic cryptosystems

Additively homomorphic cryptosystems

Application to e-voting

Somewhat fully homomorphic encryption

Fully homomorphic encryption

Somewhat fully homomorphic encryption

- Can do both homomorphic additions and multiplications, but not an arbitrary number of them

Outline

Definition and properties

Multiplicatively homomorphic cryptosystems

Additively homomorphic cryptosystems

Application to e-voting

Somewhat fully homomorphic encryption
- Pairing-based construction
- Lattice-based construction

Fully homomorphic encryption

Outline

Definition and properties

Multiplicatively homomorphic cryptosystems

Additively homomorphic cryptosystems

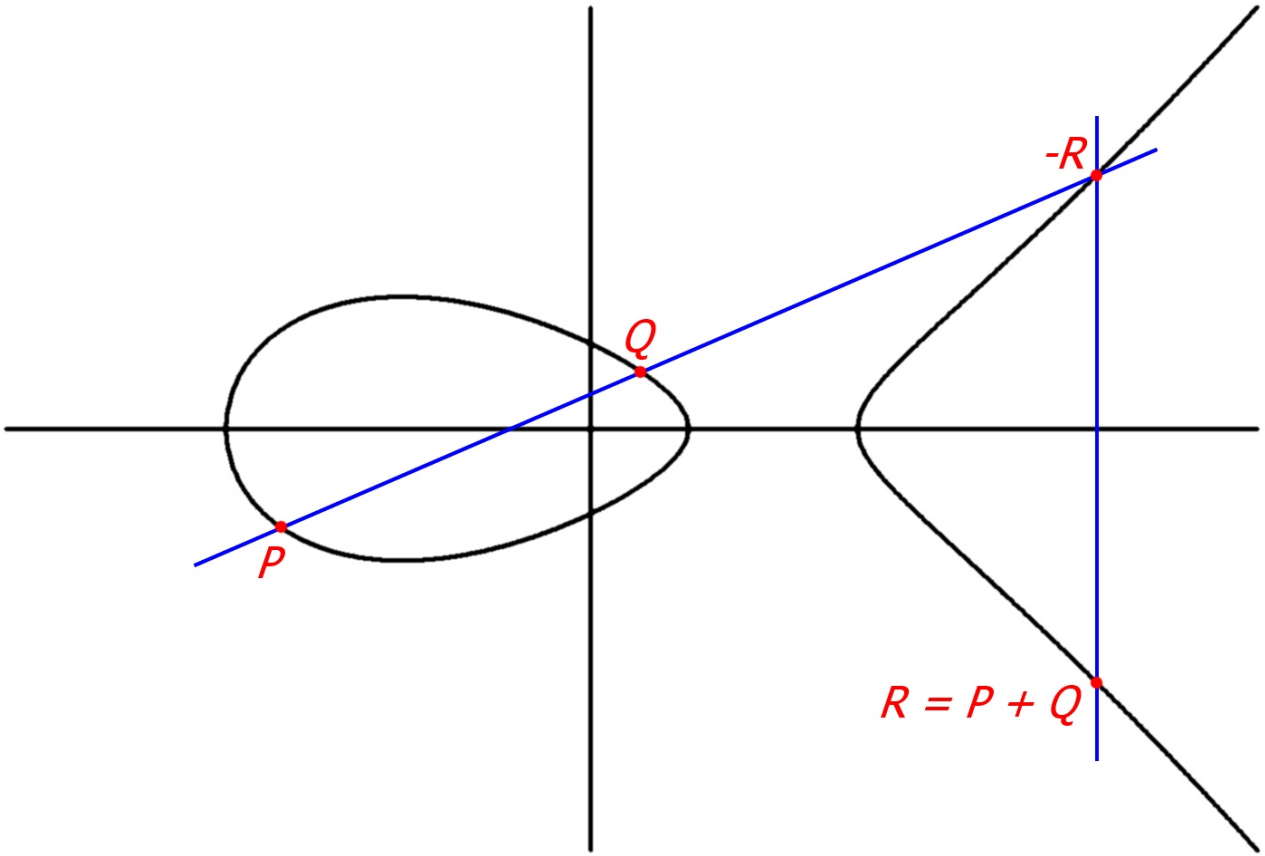
Application to e-voting

Somewhat fully homomorphic encryption
- Pairing-based construction
- Lattice-based construction

Fully homomorphic encryption

Remember: elliptic curves

$y^2 = x^3 + Ax + B.$



Remember: scalar multiplication

$$y^2 = x^3 + Ax + B.$$

- For $k \in \mathbb{Z}$ and any $P \in E(K)$, define

$$[k](P) := \underbrace{P + P + \ldots + P}_{k \text{ times}}$$

- If K finite, then for any $P \in E(K)$, there is $m \in \mathbb{Z}$ such that $[m](P) = O$ (smallest such m called *order* of P)

Elliptic curve discrete logarithm problem (ECDLP)

- Let K be a finite field, let E be an elliptic curve over K, let $P \in E(K)$ and let $Q \in \langle P \rangle = \{P, [2]P, [3]P, \ldots, O\}$. Find k such that $Q = [k]P$.
- Fields used in cryptography are prime fields (mostly) and binary fields with a prime degree extension (for efficiency, particularly in hardware devices, though less and less)
- Typically the order of P is a large prime, at least 256 bits
- Most DLP-based protocols have an ECDLP-based version

Cryptographic pairings

- Pairings are non-degenerate, bilinear maps

$$e : G_1 \times G_2 \rightarrow G_3$$

  where G_i are all cyclic groups of the same order n (usually consider G_1, G_2 additive and G_3 multiplicative)
- Bilinear:

$$\begin{aligned} e(P_1 + Q_1, P_2) &= e(P_1, P_2)e(Q_1, P_2) \\ e(P_1, P_2 + Q_2) &= e(P_1, P_2)e(P_1, Q_2) \end{aligned}$$

- Non-degenerate: for all $P_1 \in G_1$, $P_1 \neq O$, there exists $P_2 \in G_2$ such that $e(P_1, P_2) \neq 1$ (and vice-versa)

Pairing properties

- $e(jP, Q) = e(P, Q)^j = e(P, jQ)$ for any $j \in \mathbb{N}$
- $e(P, O) = e(O, P) = 1$
- $e(-P, Q) = e(P, Q)^{-1} = e(P, -Q)$
- $e(jP, Q) = e(P, Q)^j = e(P, jQ)$ for any $j \in \mathbb{Z}$

- We say the pairing is *symmetric* if $G_1 = G_2$
- For symmetric pairings

$$e(P_1, P_2) = e(k_1 P, k_2 P) = e(P, P)^{k_1 k_2} = e(P_2, P_1)$$

- Many (well-chosen) elliptic curves come with suitable (symmetric) pairings

Boneh-Goh-Nissim cryptosystem

- Let p, q be large primes and $n = pq$ as in RSA
- Let $r = 4n\ell - 1$ prime for some small ℓ
- Consider the curve $y^2 = x^3 + x$ over $\mathbb{F}_r$
  - Supersingular curve, order $\#E(\mathbb{F}_r) = r + 1 = 4n\ell$
  - Let P be a point of order n and let $G_1 = \langle P \rangle$
  - Let $G_3 = \mathbb{F}_r^*$
  - There is a symmetric bilinear pairing $e : G_1 \times G_1 \to G_3$
- Let $Q' \in G_1$ and let $Q = qQ'$ of order p
- Secret key is q, public key is (n, P, Q)

Boneh-Goh-Nissim cryptosystem

- Encryption of m
  - Choose t randomly in $[1, n]$
  - Let $C = mP + tQ$
- Decryption of C
  - Compute pC and pP
  - Compute the discrete log of pC with respect to pP
- Correctness: $pC = mpP + tpQ = mpP + tpqQ' = mpP$
- Note: decryption efficient iff m only takes few values, as in exponential El Gamal

Boneh-Goh-Nissim cryptosystem

- Homomorphic additions

$$\begin{aligned}\mathbf{Enc}(m_1) + \mathbf{Enc}(m_2) &= (m_1 P + t_1 Q) + (m_2 P + t_2 Q) \\ &= (m_1 + m_2)P + (t_1 + t_2)Q\end{aligned}$$

- "Homomorphic multiplications": compute

$$\begin{aligned}e(C_1, C_2) &= e(m_1 P + t_1 Q, m_2 P + t_2 Q) \\ &= e(P, P)^{m_1 m_2} e(P, Q)^{m_1 t_2 + m_2 t_1} e(Q, Q)^{t_1 t_2} \\ &= e(P, P)^{m_1 m_2} e(P, Q')^{(m_1 t_2 + m_2 t_1)q} e(Q', Q')^{t_1 t_2 q^2}\end{aligned}$$

- Decryption after homomorphic multiplication
  - Compute discrete logarithm of $e(C_1, C_2)^p$ with respect to $e(P, P)^p$
  - Indeed $e(C_1, C_2)^p = e(P, P)^{p m_1 m_2}$

Boneh-Goh-Nissim cryptosystem

- Homomorphic multiplications change the ciphertext group from G_1 to G_3
- Can perform any number of homomorphic additions, followed by a single homomorphic multiplication, followed by any number of homomorphic additions
- In other words: can compute the encryption of any quadratic function of plaintexts
- Note decryption more expensive if result is larger

Outline

Definition and properties

Multiplicatively homomorphic cryptosystems

Additively homomorphic cryptosystems

Application to e-voting

Somewhat fully homomorphic encryption
- Pairing-based construction
- Lattice-based construction

Fully homomorphic encryption

Symmetric key version

- Secret key is a large prime p
- To encrypt a bit m, choose random $r << p$ and large q, then return $c = m + 2r + pq$
- To decrypt c, compute $m' = (c \bmod p) \bmod 2$
- Correctness:
  $m' = ((m+2r+pq) \bmod p) \bmod 2 = (m+2r) \bmod 2 = m$
  (as long as $m + 2r < p$)
- Security related to **approximate gcd problem**
  (hard to compute p from several samples $pq_i + s_i$)
  (can be reformulated as a lattice problem)

Additively homomorphic properties

$$\begin{aligned}\mathbf{Enc}(m_1) + \mathbf{Enc}(m_2) &= (m_1 + 2r_1 + pq_1) + (m_2 + 2r_2 + pq_2) \\ &= (m_1 + m_2) + 2(r_1 + r_2) + p(q_1 + q_2)\end{aligned}$$

More generally

$$\mathbf{Enc}(\sum m_i) = \sum m_i + 2\sum r_i + p\sum q_i$$

will decrypt to $\sum m_i$ as long as $\sum m_i + 2\sum r_i < p$

Multiplicatively homomorphic properties

$$\begin{aligned}\mathbf{Enc}(m_1) \cdot \mathbf{Enc}(m_2) &= (m_1 + 2r_1 + pq_1) \cdot (m_2 + 2r_2 + pq_2) \\ &= (m_1 \cdot m_2) + 2(m_1 r_2 + r_1 m_2 + 2r_1 r_2) \\ &\quad + p(\ldots)\end{aligned}$$

More generally

$$\mathbf{Enc}(\prod m_i) = (\prod m_i) + 2R + pY$$

will decrypt to $\sum m_i$ as long as $(\prod m_i) + 2R < p$

Noise growth

- Every homomorphic operation makes the noise r grow
- Growth is much faster for multiplications than additions
- When the noise is too big ($2R > p$) the decryption algorithm may fail

- This still allows to homorphically evaluate *some* functions on encrypted data (functions with few multiplications)
- With clever tricks (suitable data representation, circuit) this suffices in many applications

Public key version

- Public key has several encryptions of 0

$$c_i = 2r_i + pq_i$$

- Encryption of m is

$$c = m + \sum_{i \in I} c_i + 2r$$

  for random subset I
- Decryption and homomorphic operations as before

Variants

- Many variants of the above scheme in literature
- All of them based on *lattice problems* (see Week 6) or can be reformulated as such
- A few specific schemes have been broken, but others based on the ("well-studied") learning with errors problem

Outline

Definition and properties

Multiplicatively homomorphic cryptosystems

Additively homomorphic cryptosystems

Application to e-voting

Somewhat fully homomorphic encryption

Fully homomorphic encryption

Fully homomorphic encryption (FHE)

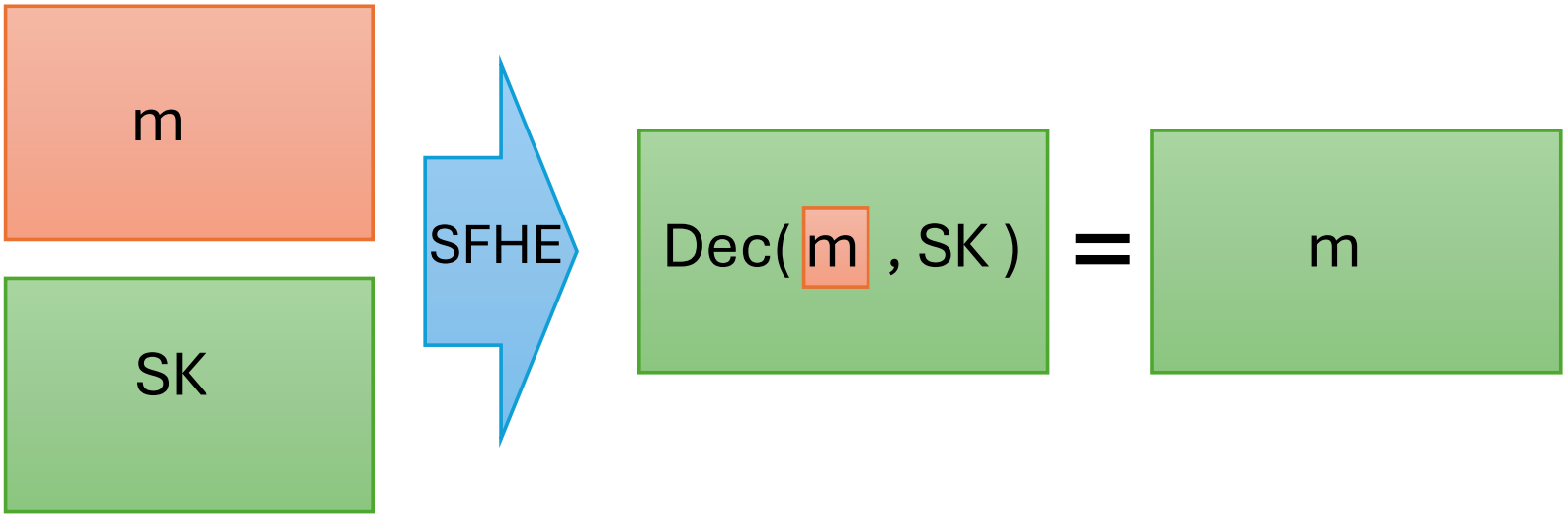
- RSA is multiplicatively homomorphic:
  $\mathrm{Enc}(m_1 m_2) = (m_1 m_2)^e \bmod n = \mathrm{Enc}(m_1)\mathrm{Enc}(m_2)$
- Additively homomorphic schemes have also been known for a long time $\mathrm{Enc}(m_1 + m_2) = \mathrm{Enc}(m_1) + \mathrm{Enc}(m_2)$
- Some schemes allow more general (but not fully general) homomorphic operations
- Satisfying both properties simultaneously allows cool stuff, such as statistics on encrypted data

- FHE was long-standing open problem until 2009
  First solution by Gentry, followed by many other ones

FHE key ideas

- Encrypt your messages as noisy ring elements like in previous encryption scheme based on lattices
- This gives **somewhat fully homomorphic encryption** Homomorphic additions and multiplications of ciphertexts, but not too many as each operation increases the noise (hence at some point you cannot decrypt correctly anymore)
- Could decrease the noise by decrypting and re-encrypting, but that would reveal intermediary plaintexts
- **Bootstrapping:** include encrypted version of somewhat homomorphic encryption decryption key in the public key, and regularly decrease noise with homomorphic decryption

Boostrapping idea (1/2)

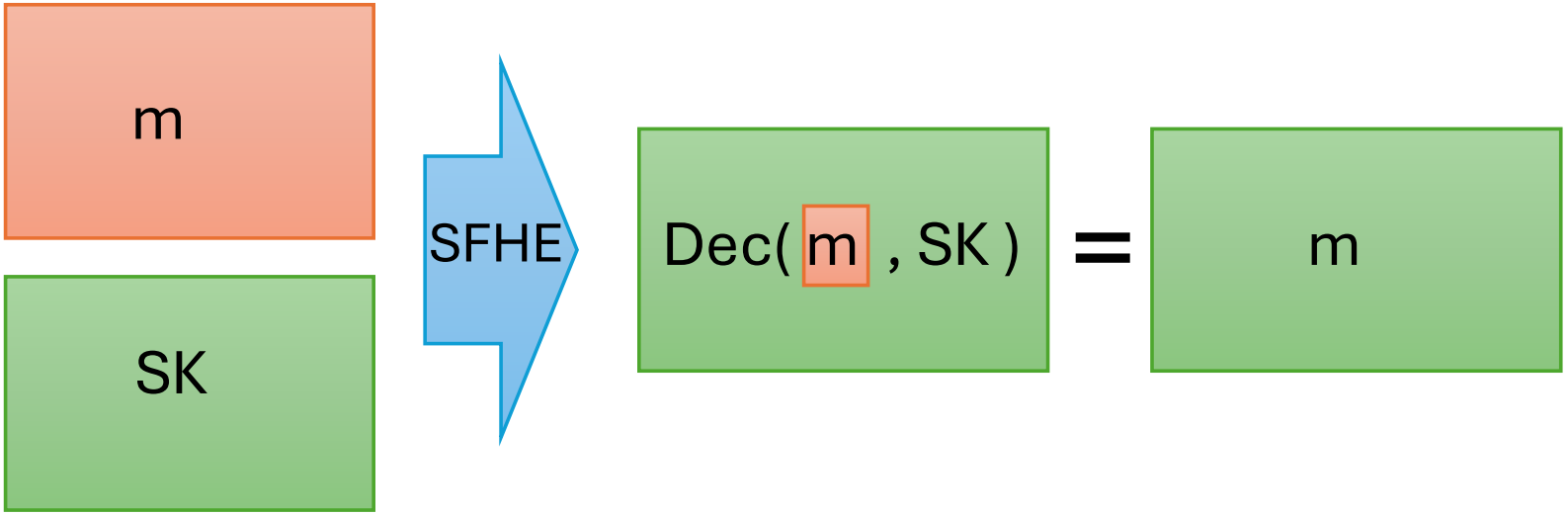


Green vs orange boxes: encryptions with small vs high noise levels

- Use SFHE to homomorphically evaluate encryption of $f_c(SK) = \mathrm{Dec}(c, SK)$ from c and encryption of SK

  E.g.: homomorphically compute $\mathrm{Enc}((c \bmod p) \bmod 2) = \mathrm{Enc}(m)$ from c and $\mathrm{Enc}(p)$, where Enc is above SFHE scheme

Boostrapping idea (2/2)



Green vs orange boxes: encryptions with small vs high noise levels

- Suppose that SFHE noise small enough after evaluating ($f_c(SK)$ + one additional multiplication) homomorphically
- Then iterate this to evaluate any circuit homomorphically!

Fast recent progress

- First implementation of Gentry's idea: terabytes keys, one homomorphic evaluation took 30s
- Considerable "engineering" progress in following years (2nd, 3d, 4th FHE generations)
  - Better ciphertext size/ plaintext size ratio
  - Slower growth rate, especially for multiplications
  - Dedicated circuits for interesting applications such as machine learning
- Many implementations HELib, SEAL, etc
- Ongoing standardization effort `http://homomorphicencryption.org/`

Outline

Definition and properties

Multiplicatively homomorphic cryptosystems

Additively homomorphic cryptosystems

Application to e-voting

Somewhat fully homomorphic encryption

Fully homomorphic encryption

Conclusion

- Many "textbook" public key cryptosystems come with some additive or multiplicative homomorphic properties
- These properties break IND-CCA security, but can also be useful in particular contexts (e-voting, . . .)
- Somewhat fully homomorphic schemes allow more operations: one multiplication and unlimited additions, or a limited number of operations
- Gentry: get fully homomorphic encryption by combining a somewhat fully homomorphic scheme and a bootstrapping technique

References

- Paillier, *Public-Key Cryptosystems Based on Composite Degree Residuosity Classes*
- Boneh-Goh-Nissim, *Evaluating 2-DNF formulas on ciphertexts*
- Silverberg, *Fully homomorphic encryption for mathematicians*
- Gentry, *A fully homomorphic encryption scheme*